
Design Review: PipelineC ChaCha20-Poly1305

1. Security

1.1 Plaintext always released regardless of verification result — no hardware gate

File: wait_to_verify.c:80-101

```
else // OUTPUT_PLAINTEXT
{
    verify_fifo_out_ready = wait_to_verify_axis_out_ready;

    wait_to_verify_axis_out.valid = verify_fifo_out.valid; // unconditional

    wait_to_verify_is_verified_out = tags_match_reg;      // 0 or 1 alongside data
```

The plaintext stream always drains, whether tags_match_reg is 0 or 1. The consumer must check is_verified_out and discard the data on failure. There is no hardware interlock inside the crypto core itself. This is a correct and common design choice (keeps the module simple and decoupled), but it creates an integration obligation that is not enforced at the port level — nothing prevents a careless integrator from ignoring the decrypt_m_verified wire. Consider asserting valid=0 on the output stream when tags_match_reg=0, which would make the module self-contained and safe regardless of integration quality.

1.2 No test for authentication failure

Files: decrypt_tb.c, encrypt_tb.c

The decrypt testbench only exercises the positive case (correct key + nonce + AAD + ciphertext + auth tag). There are no test vectors for:

- A tampered ciphertext byte (should produce is_verified=0)
- A tampered auth tag
- A mismatched AAD

For a cryptographic primitive, these failure paths are the most security-critical behavior to verify. The testbench's current coverage would pass even if the comparator in poly1305_verify_decrypt.c always returned 1.

2. Correctness

2.1 tkeep not respected in ChaCha20 — partial last blocks not RFC-compliant

Files: chacha20.h:215, encrypt_tb.c:121, decrypt_tb.c:122

The known TODO:

```
uint8_t in_data[CHACHA20_BLOCK_SIZE] = inputs.axis_in.tdata; // TODO handle tkeep
```

The testbench works around this by always asserting all 16 tkeep bits, including for the last partial block, and zero-filling unused bytes:

```
chacha20poly1305_encrypt_axis_in.data.tkeep[i] = 1; // always 1
```

```
chacha20poly1305_encrypt_axis_in.data.tdata[i] = 0;
```

```
if(plaintext_remaining > i)
```

```
    chacha20poly1305_encrypt_axis_in.data.tdata[i] = plaintext[i];
```

The consequence: both test plaintexts are 56 and 72 bytes respectively, but the ciphertext output sizes (CIPHERTEXT0_SIZE=80, CIPHERTEXT1_SIZE=96) include 8 and 8 extra zero-encrypted bytes. The expected vectors were presumably generated from a software reference that also processed 64 and 80 bytes. This means the test does not validate RFC 8439 compliance for sub-16-byte last blocks. The expected ciphertext and auth tags in the testbench are valid for the padded-to-64/80-byte messages, not the actual 56/72-byte plaintexts.

prep_auth_data.h:117 correctly uses axis128_keep_count(axis_in.data) to count actual ciphertext bytes via tkeep — but since tkeep is always forced to 1 by the testbench, this counter always gives multiples of 16. A real driver feeding partial last blocks with correct keep would produce a different auth tag than the reference because axis128_keep_count would count fewer bytes, but Poly1305 would still receive a zero-padded 16-byte block (tkeep is forced to 1 in prep_auth_data's CIPHERTEXT state). The length mismatch would corrupt the lengths block.

Fix required: Either handle partial tkeep properly end-to-end, or document explicitly that the interface contract requires the caller to always zero-pad to 16-byte multiples and present tkeep=all-1.

2.2 axis128_early_tlast returns an uninitialized struct

File: strip_auth_tag.c:27-65

```
axis128_early_tlast_t o; // no zero-init
```

```
...
```

```
if(buff_to_out_connected){  
    o.axis_out = buffer_reg;  
    o.next_axis_out_is_tlast = ...;  
}
```

```
// if not taken: o.axis_out.valid and o.next_axis_out_is_tlast are uninitialised
```

In synthesized RTL this becomes wires, so o.axis_out.valid defaults to 0 (undriven net) and the module behaves correctly in practice. However in C simulation o contains garbage unless PipelineC zero-initialises locals. A one-line fix eliminates any ambiguity:

```
axis128_early_tlast_t o = {0};
```

2.3 Misleading "TODO AAD is unused" comment

File: prep_auth_data.c:37

```
// TODO AAD is unused, could be removed
```

```
uint8_t PPCAT(PREP_AUTH_DATA_INST,_aad)[AAD_MAX_LEN];
```

```
uint8_t PPCAT(PREP_AUTH_DATA_INST,_aad_len);
```

The aad and aad_len global wires are passed to prep_auth_data_fsm and are actively used to build the authenticated data stream (the AAD_STATE branch in prep_auth_data.h is live). The testbench confirms this with a 29-byte AAD. The comment is simply wrong and should be removed before it causes confusion in future maintenance.

3. Backpressure asymmetry in shared Poly1305 (commented-out code)

File: poly1305_pipeline_shared.c:40-44

```
poly1305_mac_encrypt_compute_in_ready = 1; // unconditional
```

```
poly1305_mac_decrypt_compute_in_ready = 1; // unconditional
```

```
// poly1305_mac_compute_in_ready; // correct form, commented out
```

Compare with the ChaCha20 sharing mux which correctly routes the pipeline's actual ready signal back to the selected consumer. If the Poly1305 sharing code is ever uncommented to form a "Full Sharing" build, data will be silently dropped when the compute module is not ready. The correct back-pressure line is present but commented out. If this code path is not intended for use, it should either be deleted or fixed before being re-enabled.

4. Design Constraints

4.1 AAD capped at 32 bytes — not documented at the interface

File: prep_auth_data.h:7

```
#define AAD_MAX_LEN 32
```

This is a hard constraint on the hardware. RFC 8439 allows arbitrary-length AAD. For WireGuard specifically, the Data packet header (type

+ reserved + receiver index + counter) is 16 bytes, so 32 bytes is sufficient. But the constraint is buried in a header file and is not surfaced as a port comment, a compile-time assertion, or a runtime-checked register. An integrator presenting 33+ bytes would silently truncate the AAD and compute a different auth tag. This should be enforced or at minimum clearly documented at the top of prep_auth_data.h and at the port declaration in chacha20poly1305_encrypt.h / chacha20poly1305_decrypt.h.

4.2 Single packet in flight throughout the pipeline

The design processes one packet at a time across all modules (ChaCha20 FSM, Poly1305 MAC FSM, and wait_to_verify). This is correct and

matches the paper's one-in-flight characterisation. However it means there is no pipeline fill during inter-packet gaps — the next packet cannot start entering ChaCha20 until the previous packet's Poly1305 computation and verification have fully completed and the FSMs have reset. For bursty short-packet traffic this creates dead cycles that are not counted in the paper's throughput numbers, which only model the steady-state single-packet case.

4.3 No in-flight abort or key-rotation mechanism

None of the FSMs have an explicit reset or flush input. If the surrounding system needs to rotate keys, detect a bad packet mid-stream, or recover from a timeout, there is no mechanism to abort a partially-processed packet cleanly. The only recourse is a global FPGA reset, which also clears the FIFO. For a production WireGuard node this becomes relevant during session rekeying while packets are in flight.

5. Code Quality

5.1 Testbench does not vary the nonce between packets

Files: `encrypt_tb.c:20-22`, `decrypt_tb.c:20-22`

```
uint8_t nonce[CHACHA20_NONCE_SIZE] = { 0x07, 0x00, 0x00, 0x00, 0x40, 0x41, 0x42, 0x43, 0x44, 0x45, 0x46, 0x47};
```

Both test packets use the same (key, nonce) pair. In WireGuard the 64-bit counter embedded in the nonce is incremented per packet. Reusing a nonce with the same key is a catastrophic security failure. The testbench should at minimum vary the counter field between packets and verify that the output auth tags differ.

5.2 Error reporting uses printf with no simulation pass/fail signal

The testbench prints "ERROR: ..." strings on mismatch but has no dedicated pass/fail wire that the cocotb harness can latch. If the simulation runner doesn't parse stdout, test failures are invisible at the CI level. A \$display/assertion-based pass/fail flag would be more robust.

5.3 poly1305_verify_decrypt.c has a redundant COMPARE_TAGS state

```
else if (state == COMPARE_TAGS)
{
    tags_match_reg = (auth_tag_reg == calc_tag_reg);
    state = OUTPUT_COMPARE_RESULT;
}
```

COMPARE_TAGS takes exactly one cycle and has no I/O. It exists only to latch the comparison result before outputting it, which is correct, but the same can be achieved by merging the comparison into TAKE_CALC_TAG on the accepting cycle, saving one state transition and one cycle of latency. This is minor but in a tight Poly1305 feedback loop every cycle counts.

5.4 append_auth_tag.c file header says "Append the auth tag" — same as strip_auth_tag.c

```
// Append the auth tag after the ciphertext ← strip_auth_tag.c line 6
```

Copy-paste error in the file header comment of strip_auth_tag.c. Minor, but confusing.

6. Improvement Suggestions — Prioritised

Priority	Suggestion
High	Gate the wait_to_verify output to valid=0 when tags_match_reg=0 so plaintext is suppressed in hardware, removing the consumer obligation
High	Add auth-failure test vectors: tampered ciphertext, tampered tag, wrong AAD – these are the cases that actually exercise the security function
High	Resolve the tkeep partial-block handling: either implement it end-to-end (correct axis128_keep_count inputs, correct ChaCha20 masking) or document the 16-byte-aligned-only contract at the top-level ports
Medium	Add a flush/abort input to the top-level that resets all FSMs and clears the FIFO cleanly without a global reset
Medium	Add a compile-time <code>_Static_assert(AAD_MAX_LEN == 32, ...)</code> and document the constraint prominently in the port headers
Medium	Fix the axis128_early_tlast struct to zero-initialise <code>o</code>
Medium	Vary nonce between testbench packets and assert that output auth tags differ
Low	Remove the incorrect "TODO AAD is unused" comment in <code>prep_auth_data.c</code>
Low	Fix the copy-paste file header in <code>strip_auth_tag.c</code>
Low	Merge <code>COMPARE_TAGS</code> into <code>TAKE_CALC_TAG</code> to save one FSM cycle in the verify path
Low	Add a cocotb-observable pass/fail signal rather than relying on stdout parsing
Future	Implement packet-level parallelism (1-to-N Poly1305) to approach 10G line rate as described in the paper's future work section